PES UNIVERSITY
(Established under Karnataka Act No. 16 of 2013)
Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

A Project Report
On

AI-Assisted 3D Assembly Design

Predicting Missing Components in CAD Assemblies using Graph Neural Networks

Submitted in fulfilment of the requirements for
Project Phase – 1

Submitted by

Parthasarathy Perumal
SRN: PES2PGE24DS193

Under the guidance of
Prof. Sagarika Borah
Professor, Dept. of Computer Science and Engineering

May – July 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
FACULTY OF ENGINEERING
PROGRAM: M.TECH – DATA SCIENCE & ARTIFICIAL INTELLIGENCE

CERTIFICATE

This is to certify that the project report entitled **"AI-Assisted 3D Assembly Design: Predicting Missing Components in CAD Assemblies using Graph Neural Networks"** is a bonafide work carried out by **Parthasarathy Perumal (SRN: PES2PGE24DS193)** in partial fulfilment for the completion of Project Phase – 1 in the Program of Study, Master of Technology in Data Science & Artificial Intelligence, under the rules and regulations of PES University, Bengaluru, during the period May 2026 – July 2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of project work for this semester.

Signature with date	Signature with date	Signature with date
Internal Guide	**Chairperson**	**Dean of Faculty**
Prof. Sagarika Borah		

Name and Signatures of the Examiners

1.

2.

DECLARATION

I hereby declare that the Project Phase – 1 entitled **"AI-Assisted 3D Assembly Design: Predicting Missing Components in CAD Assemblies using Graph Neural Networks"** has been carried out by me under the guidance of Prof. Sagarika Borah, Department of Computer Science and Engineering, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science & Artificial Intelligence of PES University, Bengaluru, during the academic period May – July 2026. The matter embodied in this report has not been submitted to any other university or institution for the award of any degree.

SRN: PES2PGE24DS193 Parthasarathy Perumal

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Prof. Sagarika Borah, Department of Computer Science and Engineering, PES University, for her continuous guidance, feedback, and encouragement throughout this project. Her suggestion during the First Review to strengthen the geometric grounding of the feature pipeline, and her published work on Octree-based spatial partitioning for polygon mesh analysis, directly shaped two of the key modules described in this report – the geometry-driven node feature enrichment and the open surface detection module.

I am also thankful to the project review panel and coordinators at PES University for their time and constructive comments during the zeroth, first, second, and third review sessions, which helped correct course at several points during the project. Finally, I extend my thanks to my family and friends for their patience and support through the course of this work.

ABSTRACT

Engineering CAD assemblies consist of multiple interconnected components whose correct selection is time-consuming and expertise-dependent. Modern CAD tools such as SolidWorks, CATIA, and Fusion 360 provide no contextual intelligence during assembly creation, forcing engineers to rely entirely on accumulated domain knowledge to choose and position components. This project addresses that gap by training a **Graph Attention Network (GAT)** on assembly graphs derived from STEP files to detect **missing components** via link prediction.

Each CAD assembly is converted into an attributed graph using gmsh with the OpenCASCADE kernel: solid bodies become nodes and physical contacts between bodies become edges. The node feature vector was progressively enriched over the course of the project from a 13-dimensional bounding-box-only representation to a 22-dimensional vector that includes geometry-driven component typing (via Shape Diameter Function ray-casting), exact mesh surface area, affine-invariant shape descriptors, and hole-count metadata; the edge feature vector was similarly expanded from 2 to 6 dimensions to include joint-type information. A 3-layer GAT encoder produces 64-dimensional node embeddings, which are passed to a Link Predictor MLP that scores candidate node pairs to identify missing component connections.

The system incorporates three complementary analysis modules beyond the core GNN: (1) an **Assembly Completeness Model** (`AssemblyTemplateDB`) that learns per-category component-type distributions from training data to identify what is missing even from a single-part upload, (2) an **Octree-based Open Surface Detector**, adapted from the spatial partitioning technique in Borah & Borah (2020), that localises the precise mesh surfaces where a missing component should be assembled, and (3) a **Gemini-powered Skills AI agent** ("AIDA") that translates the numerical outputs of the above into structured engineering-language explanations.

Across 25 documented training iterations (R1–R28) spanning dataset scale-ups, feature engineering passes, and category-focused ablations, the most stable result was obtained on a deduplicated, quality-filtered corpus of 1,760 assembly graphs (643 from the Fusion 360 Gallery dataset and 137 curated local assemblies): a mean AUC-ROC of 0.625 ± 0.017 and mean Average Precision of 0.878 ± 0.005 under 5-fold cross-validation. The currently deployed model, trained on a curated 695-graph, very-high-connectivity Mechanical Engineering subset, achieves mean AUC-ROC 0.546 ± 0.025 and mean AP 0.712 ± 0.014, continuing an upward AUC trend across successive category-focused promotions (R23 $\rightarrow$ R27 $\rightarrow$ R28: $0.460 \rightarrow 0.531 \rightarrow 0.546$). Average Precision consistently exceeds the Phase 1 target of 0.82 across multiple runs; AUC-ROC remains below the 0.85 target, motivating the heterogeneous-GNN and ranking-based improvements planned for Phase 2. The solution is implemented entirely in Python using PyTorch Geometric, without dependency on any proprietary CAD software API, and is deployed as an interactive Streamlit application with real-time 3D visualisation.

Table of Contents

List of Figures

List of Tables

1 Introduction

Computer-Aided Design (CAD) is the cornerstone of modern engineering, enabling the creation, modification, and optimisation of designs in a digital environment. CAD assemblies – collections of multiple interconnected mechanical components – are central to industries ranging from automotive to aerospace, consumer electronics, and industrial machinery. Assembling components in a CAD environment requires selecting the right parts, positioning them correctly, and defining the appropriate mate constraints (coincident, concentric, tangent, and so on) between them.

Despite decades of CAD tool evolution, the assembly process remains largely manual and expertise-dependent. Engineers rely on years of accumulated domain knowledge to identify which components belong in an assembly, to notice when parts are missing, and to determine an appropriate assembly sequence. This creates practical difficulties: junior engineers face a steep learning curve, different designers make inconsistent choices for equivalent subassemblies, and a partially defined assembly has no built-in mechanism to flag or fill missing parts.

This project explores the use of **Graph Neural Networks (GNNs)** for intelligent assembly assistance. By representing a CAD assembly as a graph – solid bodies as nodes, physical contacts between bodies as edges – the problem of detecting a missing component maps naturally onto **link prediction**: identifying an edge that should exist between two nodes but is currently absent.

1.1 Background

An assembly graph built directly from raw STEP geometry, without any proprietary CAD-vendor metadata, only has access to what can be derived purely from geometry: volumes, surface areas, bounding boxes, and detected physical contacts between solids. This geometry-only constraint was adopted deliberately so that the resulting pipeline works identically on STEP files exported from SolidWorks, CATIA, Fusion 360, or any other ISO 10303-compliant CAD system, rather than being tied to one vendor's proprietary assembly-tree format.

1.2 Problem Statement

Modern CAD tools offer no contextual intelligence during assembly creation. Engineers rely entirely on domain knowledge to choose and position each component. The key pain points are:

1. **Knowledge barrier** – junior engineers lack the assembly patterns that experienced designers build up over years of practice.
2. **No automation** – existing CAD tools offer no intelligent part suggestion or missing-component detection during assembly creation.
3. **Non-standardisation** – different designers make inconsistent choices for structurally equivalent subassemblies, creating quality and maintenance issues downstream.
4. **Incomplete assemblies** – a partially defined assembly has no built-in mechanism to detect or fill missing parts automatically.

Research gap: no existing system applies graph-based deep learning to recommend components from a partial CAD assembly graph without depending on a proprietary CAD API.

1.3 Objectives

1. **Graph construction pipeline** – build a robust pipeline that converts STEP files into attributed assembly graphs, using gmsh (OpenCASCADE) with geometry-enriched node and edge features.
2. **Missing component detection** – train a Graph Attention Network (GAT) with a Link Predictor head to detect missing connections in a partial assembly, targeting AUC-ROC ≥ 0.85 and Average Precision ≥ 0.82.
3. **Assembly completeness model** – build a template-based system that learns per-category component-type distributions and identifies which component types are missing from a partial assembly.
4. **Open surface detection** – implement a spatial analyser that localises the precise physical surfaces where a missing component should be attached.
5. **AI-powered explanation** – integrate a Gemini-based Skills AI agent that translates raw GNN scores into actionable engineering language.
6. **Interactive deployment** – ship the system as a Streamlit web application supporting upload, 3D visualisation, inference, and explanation in a single workflow.

1.4 Scope of the Project

The project is planned across two phases:

- **Phase 1 (May – July 2026, this report):** base replication – missing-component detection via GAT link prediction, the assembly completeness model, open surface detection, the AIDA Skills AI agent, and Streamlit deployment. Evaluation metrics: AUC-ROC and Average Precision.
- **Phase 2 (July – September 2026, planned):** novelty and improvement – next-component ranking via a `NodeRanker` head (cosine similarity over candidate embeddings), a heterogeneous GNN with typed node/edge parameters, a BPR ranking loss, and GNNExplainer-based interpretability. Evaluation metrics: Hit@K, MRR, NDCG@K.

The system operates on standard STEP/STP files (ISO 10303) and does not require any proprietary CAD software API, making it portable across CAD platforms.

1.5 Organisation of the Report

- **Chapter 2 – Literature Survey** reviews existing research in GNN-based geometric learning, CAD representation learning, and mesh spatial analysis.
- **Chapter 3 – System Requirements Specification** details hardware, software, functional, and non-functional requirements.
- **Chapter 4 – Proposed Methodology** describes the system architecture, the graph construction pipeline, the model architecture, and the complementary analysis modules.
- **Chapter 5 – Implementation Details** covers the development environment, dataset processing, and evaluation approach.

- **Chapter 6 – Results and Discussion** presents results across 25 documented training iterations.
- **Chapter 7 – Conclusion and Future Work** summarises the outcomes of Phase 1 and outlines the Phase 2 plan.

2 Literature Survey

This chapter reviews research relevant to the project, spanning graph neural networks, B-Rep generative models, CAD representation learning, and spatial mesh analysis, and identifies the gaps that this project sets out to address.

2.1 Graph Neural Networks for Link Prediction

Kipf & Welling (2017) introduced the Graph Convolutional Network (GCN), which applies spectral convolutions to graph-structured data by aggregating neighbour features through a normalised adjacency matrix. GCN performs well on node classification and link prediction benchmarks but treats all neighbours with equal importance, lacking a mechanism to weight the relative significance of different connections.

Veličković et al. (2018) proposed the Graph Attention Network (GAT), which introduces learnable attention coefficients into the neighbourhood-aggregation step. Multi-head attention allows the model to attend to several structural patterns simultaneously. GAT has been shown to outperform GCN on heterogeneous graphs where neighbour importance varies – a property directly relevant to assembly graphs, where different mate types (coincident, concentric, tangent) carry different structural significance.

Hamilton, Ying & Leskovec (2017) introduced GraphSAGE, which learns node embeddings by sampling and aggregating features from a node's local neighbourhood, supporting inductive learning on nodes unseen during training – relevant to inference on assemblies not present in the training corpus.

An anonymous 2024 study, "Can GNNs Learn Link Heuristics?", investigates whether GNNs can recover classical link-prediction heuristics such as common-neighbour count, the Jaccard coefficient, and the Adamic–Adar index directly from graph structure, and finds that explicit structural features can usefully complement learned representations.

2.2 B-Rep Generative Models and CAD Synthesis

Du et al. (2024), **BrepGen**, introduced a B-Rep generative diffusion model with structured latent geometry, generating boundary-representation solids by learning latent codes for faces, edges, and vertices and decoding them into valid CAD geometry. The work targets single-part generation rather than assembly analysis, but demonstrates that deep learning can capture the geometric structure of B-Rep models.

Jayaraman et al. (2024), **SolidGen**, proposed an autoregressive model for direct B-Rep synthesis and editing, generating CAD construction sequences step by step. The autoregressive framing is conceptually related to next-component prediction in assembly design, where each added component depends on the existing partial assembly – the direction planned for this project's Phase 2.

Wang et al. (2025), **CAD-GPT**, synthesises CAD construction sequences using spatial-reasoning-enhanced multimodal LLMs, combining language understanding with 3D spatial reasoning to generate models from natural-language descriptions. This highlights the poten-

tial of AI-assisted CAD workflows, but again at the individual-part rather than assembly level.

2.3 Heterogeneous Graph Learning

An anonymous 2023 paper, "Heterogeneous Graph Contrastive Learning", proposes contrastive learning for heterogeneous graphs with multiple node and edge types. This is directly relevant to CAD assemblies, where nodes represent different component types (body, fastener, bearing, shaft, plate, housing, gear) and edges represent different mate constraints (coincident, concentric, parallel, tangent, fixed). Phase 2 of this project plans to build on these ideas via a heterogeneous GNN with typed parameters.

2.4 Assembly Analysis and Spatial Reasoning

Jones et al. (2021), "Learning Bottom-up Assembly of Parametric CAD Joints", addresses the prediction of assembly joints between parametric CAD parts. This is the closest prior work to the objectives of this project, but relies on parametric CAD representations and vendor-specific joint annotations, whereas the approach adopted here works from geometry alone.

Koch et al. (2019) released the **ABC Dataset**, a collection of over one million CAD models in STEP/B-Rep format with geometric metadata such as bounding boxes, volume, and surface area – a foundational resource for geometric deep learning on CAD models.

Mo et al. (2019) contributed **PartNet**, a benchmark of 573,585 3D parts across 26 object categories with fine-grained, instance-level, and hierarchical part annotations. PartNet's hierarchical annotation style informed the edge-feature construction approach used in this project.

Willis et al. (2021) released the **Fusion 360 Gallery Dataset**, containing 8,251 assemblies and 154K bodies extracted from the Autodesk Fusion 360 platform. This project integrates 643 assembly STEP files from the Fusion 360 Gallery as the largest single component of its training corpus.

2.5 Mesh Processing and Spatial Partitioning

Borah & Borah (2020) proposed a Prediction Error Expansion (PEE) based reversible polygon-mesh watermarking scheme that uses Octree spatial partitioning for regional tamper localisation. An Octree divides the mesh's bounding volume into independent spatial sub-blocks, and each block's vertices are authenticated separately so that tampering can be localised to a region rather than merely detected globally. This spatial-decomposition principle – localised, per-region decisions from a global structure – is directly adapted in this project's open surface detection module, where an Octree partitions free-surface centroids to localise where a missing component should be placed.

Ying et al. (2019), **GNNExplainer**, introduced a model-agnostic method for explaining GNN predictions by identifying the subgraph structures and node features most responsible for a given prediction. GNNExplainer is planned for Phase 2 integration to complement AIDA's natural-language explanations with structural evidence.

2.6 Summary and Limitations of Existing Systems

The reviewed literature reveals five recurring limitations that this project's design directly responds to:

1. **Part-level focus** – most generative CAD models (BrepGen, SolidGen, CAD-GPT) operate at the level of an individual part, not at the assembly level where component interactions matter.
2. **Proprietary API dependency** – systems such as the Fusion 360 joint-prediction work rely on vendor-specific CAD APIs for constraint metadata, limiting portability across CAD platforms.
3. **No geometry-driven component typing** – existing approaches typically require a labelled component-type dataset, rather than inferring type directly from geometric signal.
4. **No spatial localisation of the gap** – no reviewed system identifies the precise physical surface where a missing component should be placed; they only name what type is missing.
5. **No engineering-language explanation** – predictions are typically presented as raw numerical scores rather than translated into an actionable recommendation.

3 System Requirements Specification

This chapter specifies the hardware and software environment, and the functional and non-functional requirements the system must satisfy.

3.1 Introduction and Project Scope

The system accepts a STEP/STP assembly file as input, analyses it through a geometry-parsing pipeline, a trained GAT-based link predictor, a template-matching module, and a spatial open-surface detector, and returns a structured, colour-coded analysis together with a natural-language explanation, rendered inside an interactive 3D viewer. Training a new model on a fresh corpus of STEP files, and running inference against an already-trained model, are both first-class, user-triggerable workflows in the same application.

3.2 Functional Requirements

ID	Requirement
FR1	Parse STEP/STP files (ISO 10303) and extract solid bodies, volumes, surface areas, bounding boxes, and surface-contact topology.
FR2	Convert a parsed assembly into an attributed PyG graph with geometry-derived node and edge features.
FR3	Train a 3-layer GAT encoder with a Link Predictor head using k-fold cross-validation, with hyperparameters configurable via a single YAML file.
FR4	Predict missing connections between assembly components and report the top-K candidates with confidence scores.
FR5	Identify the assembly category (e.g. Hinge Assembly, Shaft + Bearing + Housing) from an uploaded STEP file, with a confidence score.
FR6	Compare an uploaded assembly against learned per-category templates to identify which component types are missing.
FR7	Detect and visualise unmated surfaces where a missing component should attach.
FR8	Generate a structured engineering-language explanation of the predictions using the Gemini-based Skills AI agent.
FR9	Render an uploaded assembly in an interactive 3D viewer with colour-coded highlights for each analysis category.
FR10	Support single-body uploads by inferring the component type geometrically, matching it to an assembly category, and reporting the components expected to complete that assembly.

3.3 Non-Functional Requirements

ID	Requirement
NFR1	STEP parsing and inference shall complete within 60 seconds for assemblies with ≤ 20 bodies.

NFR2	The system shall remain tractable for assemblies ranging from 2 to 448 bodies (the observed dataset range) via configurable graph-size filters.
NFR3	No dependency on a proprietary CAD API (SolidWorks, CATIA, Fusion 360) – standard STEP files only.
NFR4	The Streamlit interface shall provide upload, training, and inference workflows with visual feedback (progress indicators, streamed activity log, colour-coded analysis panels).
NFR5	All experiments shall use fixed random seeds, deterministic assembly-ID-based splits, and version-tracked configuration to ensure reproducibility.
NFR6	The AIDA agent shall degrade gracefully to an offline mode when no Gemini API key is configured, and single-body uploads shall trigger a geometry-only analysis path without requiring GNN inference.

3.4 Hardware Requirements

Component	**Specification**
Processor	Apple M-series (ARM64) chip, or an equivalent x86_64 CPU with $\geq$ 8 cores
Memory (RAM)	$\geq$ 16 GB (32 GB recommended for large assemblies)
Storage	$\geq$ 50 GB SSD for dataset, checkpoints, and processed graph cache
GPU	Apple Metal Performance Shaders (MPS) for PyTorch acceleration; NVIDIA CUDA optional on x86
Display	$\geq 1920 \times 1080$ for the interactive 3D viewer

Table 1: Hardware Requirements

3.5 Software Requirements

Software	**Version / Role**
Operating System	macOS 13+ (ARM64) / Ubuntu 22.04+ / Windows 11
Python	3.10+ (developed on 3.12)
Package Manager	`uv`
PyTorch	2.x, MPS or CUDA backend
PyTorch Geometric	2.x – GATConv, BatchNorm, RandomLinkSplit, DataLoader
gmsh	4.x with OpenCASCADE kernel
trimesh	Latest – exact surface area, SDF ray-casting
Streamlit	1.x – interactive web application
Plotly	5.x – 3D mesh visualisation (`Mesh3d`)
PyVista	Latest – offscreen STL loading
scikit-learn	Latest – AUC-ROC, Average Precision
Google Generative AI SDK	`google-generativeai` – Gemini API access

Table 2: Software Requirements

3.6 Use Case Overview

The primary use cases are: (1) an engineer uploads a STEP file, and the system parses it, constructs the graph, runs inference, and displays the analysis; (2) an engineer triggers training, and the system scans the configured source directory, builds the dataset, trains the model, and reports metrics; (3) an engineer reviews the six-section analysis panel together with the colour-coded 3D overlay; (4) an engineer reads the AIDA explanation for a structured, natural-language summary of the same findings.

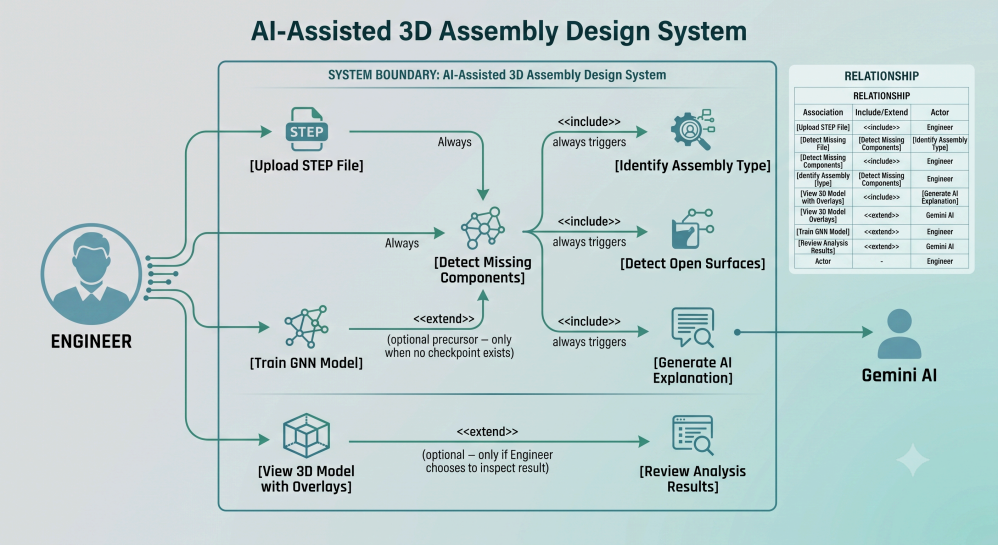

Figure 1: Use case diagram for the AI-Assisted 3D Assembly Design system.

4 Proposed Methodology

This chapter explains the system architecture, the graph construction pipeline, the model architecture, and the three complementary analysis modules that together form the system.

4.1 System Architecture

The system follows a modular pipeline with four stages: (1) STEP parsing and graph construction, (2) GNN-based link prediction, (3) assembly-completeness and open-surface analysis running alongside the GNN, and (4) AI-powered explanation and visualisation.

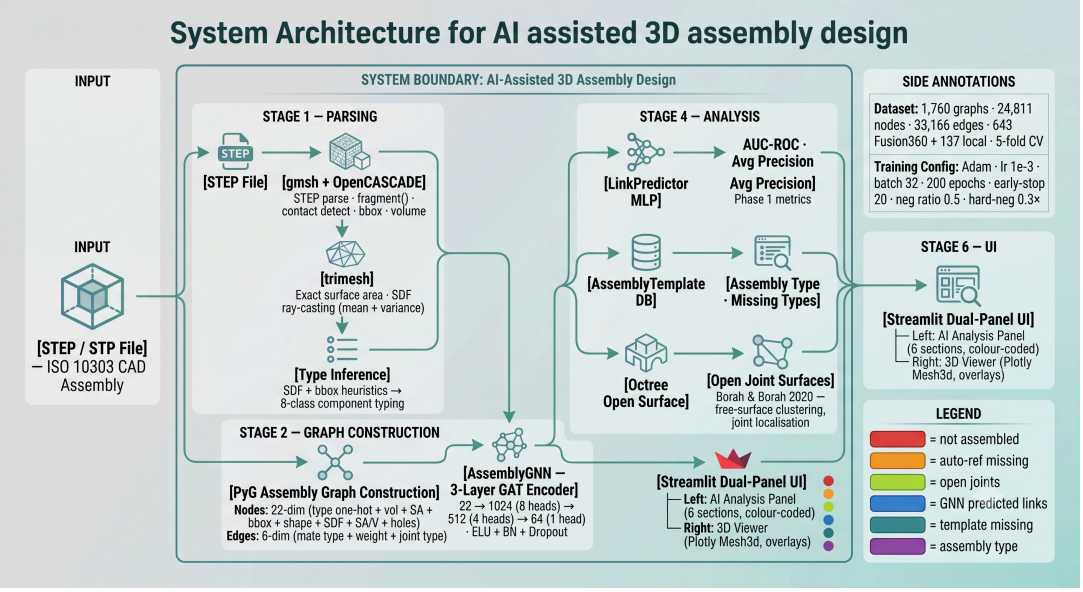

Figure 2: System architecture – from STEP file input, through graph construction and GNN inference, to the complementary analysis modules and the Streamlit front end.

At a high level:

```
STEP file --> gmsh (OpenCASCADE) --> PyG assembly graph
  Nodes (22-dim): type one-hot(8) + log-vol + log-SA
    + bbox dx/dy/dz + affine-invariant shape(5)
    + SDF mean + SDF var + SA/V + log-holes
  Edges (6-dim): mate-type encoded + weight + joint-type one-hot(4)

Assembly graph --> AssemblyGNN (3-layer GAT) --> node embeddings (64-dim)
  --> LinkPredictor (MLP) --> binary edge scores
  --> missing component detection (AUC-ROC, AP)

Parallel:  AssemblyTemplateDB --> assembly type + missing component types
           OctreeNode          --> open surface joints

Skills AI: Gemini (gemini-2.0-flash) --> AIDA structured explanation
All outputs --> Streamlit dual-panel UI (analysis panel + 3D viewer)
```

4.2 Graph Construction Pipeline

The graph construction pipeline (`back_end/dataset.py`) converts each STEP file into an attributed PyG `Data` object using a three-tool approach.

4.2.1 Step 1 – STEP Parsing with gmsh and OpenCASCADE

1. Load the STEP file and synchronise the OpenCASCADE kernel.
2. Perform **Boolean fragmentation** via `gmsh.model.occ.fragment()`. By default, STEP bodies are independent solids with no shared topology; `fragment()` performs a Boolean intersection of all volumes, forcing physically touching bodies to share the exact same boundary surface tags – without this step no shared surfaces, and therefore no edges, could be detected.
3. For each solid body, extract volume (`getMass(3, tag)`), bounding box (`getBoundingBox()`), and the set of boundary surface tags (`getBoundary()`).
4. Declare an edge between two bodies whenever their boundary surface tag sets intersect – this is the operational definition of "physical contact" used throughout the project.

4.2.2 Step 2 – Geometry Enrichment with trimesh

1. Export each body as an STL mesh via gmsh.
2. Compute the **exact surface area** using `trimesh.Trimesh.area`. This replaced the bounding-box approximation $2(dx \cdot dy + dy \cdot dz + dz \cdot dx)$ used in the earliest version of the pipeline, which was identified as the single biggest accuracy flaw in the original 13-dimensional feature vector.
3. Compute **Shape Diameter Function (SDF)** statistics via trimesh inward ray-casting: sample N surface points, cast a ray inward along the surface normal at each, record the first-hit distance (the local material thickness at that point), and aggregate across all sampled points into an SDF mean (average thickness) and SDF variance (shape complexity).

4.2.3 Step 3 – Component Type Inference

SDF statistics and bounding-box ratios drive a geometry-only classification of each body into one of eight component-type classes, replacing an earlier version of the pipeline in which every body was hardcoded to the generic "body" type.

Index	Class	SDF Signature
0	body	Generic fallback
1	fastener	Elongated ($> 2.5\times$), thin walls (SDF mean $< 8\%$ of long axis)
2	bearing	SDF std. dev. $> 60\%$ of mean – bimodal ring topology
3	shaft	Strongly elongated ($> 3.5\times$), SDF mean $> 5\%$ of long axis
4	plate	Flatness ratio $< 12\%$
5	housing	SDF variance $> 20\%$ of mean^2 – complex wall geometry
6	gear	Reserved – explicit rule planned for Phase 2
7	other	Reserved fallback

Table 3: Geometry-driven component-type classification rules.

4.2.4 Node Feature Vector

Dim	Feature	Source
0–7	Component type one-hot (8 classes)	SDF-inferred geometry rules
8	$\log(1 + \text{volume})$, clipped	`gmsh.occ.getMass()`
9	$\log(1 + \text{surface area})$, clipped	`trimesh.Trimesh.area`
10–12	Bbox $\Delta x, \Delta y, \Delta z$ / bbox_max	`getBoundingBox()`
13	Elongation (longest / mid axis)	Sorted bbox axes
14	Flatness (min / max axis)	Sorted bbox axes
15–16	Aspect x/y, aspect y/z, normalised	`getBoundingBox()`
17	Sphericity: $\pi^{\frac{1}{3}} \frac{(6V)^{\frac{2}{3}}}{\text{SA}}$	gmsh + trimesh
18–19	SDF mean, SDF variance (normalised)	Inward ray-casting
20	SA/V ratio (normalised)	trimesh + gmsh
21	$\log(1 + n_{\text{holes}})$	JSON metadata (`assembly.json`)

Table 4: The 22-dimensional node feature vector used from run R18 onward.

4.2.5 Edge Feature Vector

Dim	Feature	Description
0	Mate type	Normalised: 0.0 = coincident, 0.2 = concentric, 0.4 = parallel, 0.6 = tangent, 0.8 = fixed, 1.0 = other
1	Weight	1.0 for every detected contact
2–5	Joint-type one-hot	rigid, revolute, slider, cylindrical (from `assembly.json`)

Table 5: The 6-dimensional edge feature vector used from run R18 onward.

A current, acknowledged limitation is that mate type for every **detected** contact is hardcoded to 0.0 (coincident) and every fallback edge to 1.0 (other): the true constraint type (concentric versus tangent, for instance) is not recoverable from raw STEP boundary geometry alone – it requires CAD assembly constraint metadata that is not part of the STEP standard used here. This limitation is carried forward deliberately rather than approximated, to avoid introducing a false signal.

4.3 Model Architecture

4.3.1 AssemblyGNN – 3-Layer GAT Encoder

Layer	Input Dim	Output Dim	Heads	Notes
GAT 1	22	$128 \times 8 = 1024$	8	Edge features injected
GAT 2	1024	$128 \times 4 = 512$	4	
GAT 3	512	64	1	Single head, no concatenation

Table 6: AssemblyGNN encoder architecture. `hidden_dim` was reduced to 128 from run R17 onward to keep training tractable on the larger, 1,760-graph corpus.

Each GAT layer applies

$$\boldsymbol{h}_i^{(l+1)} = \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(l)} \boldsymbol{W}^{(l)} \boldsymbol{h}_j^{(l)}\right)$$

where the attention coefficients α_{ij} are learned as

$$\alpha_{ij} = \frac{\exp\big(\text{LeakyReLU}\big(\boldsymbol{a}^T \left[\boldsymbol{W}\boldsymbol{h}_i \parallel \boldsymbol{W}\boldsymbol{h}_j\right]\big)\big)}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\boldsymbol{a}^T [\boldsymbol{W}\boldsymbol{h}_i \parallel \boldsymbol{W}\boldsymbol{h}_k]))}$$

and K-head attention concatenates K independently-parameterised attention outputs:

$$\boldsymbol{h}_i^{(l+1)} = \Big\Vert_{k=1}^{K} \sigma\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{k} \boldsymbol{W}^k \boldsymbol{h}_j\right)$$

Each layer is followed by an ELU activation, batch normalisation, and dropout ($p = 0.2$).

4.3.2 LinkPredictor – MLP Task Head

The Link Predictor scores a candidate edge (u, v) by concatenating the two node embeddings and passing the result through a small MLP:

$$\text{score}(u, v) = \text{MLP}(\boldsymbol{z}_u \parallel \boldsymbol{z}_v) \in \mathbb{R}, \quad \boldsymbol{z}_u, \boldsymbol{z}_v \in \mathbb{R}^{64}$$

with `MLP = Linear(128 -> 64) -> ReLU -> Dropout(0.1) -> Linear(64 -> 1)`.

4.3.3 Training Procedure

Parameter	Value
Optimizer	Adam ($\text{lr} = 10^{-3}$, weight_decay $= 5 \times 10^{-4}$)
LR schedule	ReduceLROnPlateau (factor = 0.5, patience = 8)
Early stopping	Patience = 20 epochs
Max epochs	200
Batch size	32 graphs
Negative sampling ratio	0.5 per positive edge
Hard-negative weight	0.3× BCE loss
Cross-validation	5-fold, split by assembly ID
Loss function	BCE with logits + hard-negative term

Table 7: Training hyperparameters (`back_end/config.yaml`).

Hard negative sampling. For each positive-edge source node, the model locates the most similar non-neighbour node by cosine similarity of embeddings, and adds it as a hard negative with a $0.3\times$ weight in the loss. This discourages the model from relying purely on embedding similarity and forces it to discriminate genuinely connected nodes from merely similar ones.

4.4 Complementary Analysis Modules

4.4.1 Assembly Completeness Model (`AssemblyTemplateDB`)

`AssemblyTemplateDB` learns the expected component-type composition of each assembly category. During the **build phase**, run automatically after every training pass, it loads the processed graphs and their source paths, groups them by top-level source folder (for example `Hinge_assembly/`), computes the median count of each component type across all assemblies in that category, and caches the result to `assembly_templates.json`.

At inference time, the match score for a candidate category is

$$\text{score} = \frac{\sum_{t \in \text{ types}} \min(\text{present}(t), \text{expected}(t))}{\max(\sum \text{present}, \sum \text{expected})}$$

with a 25% bonus applied when every present component type fits inside the template (that is, no unexpected component types appear). The minimum confidence threshold for a match is 0.10. Given a matched template, the gap between the template's expected counts and the counts actually present in the upload gives the list of missing component types – this mechanism works even for a single-body upload, where the single body's inferred type is matched against every known template.

4.4.2 Open Surface Detection (Octree-based)

This module is adapted from the Octree spatial-partitioning concept described in Borah & Borah (2020), reused here for a structurally different purpose.

Concept	PEE Watermarking (2020)	This Project
Spatial partition	Octree of mesh vertices	Octree of free-surface centroids
Unit of analysis	Vertex block	Surface cluster
Per-block decision	Watermark valid / tampered	Surface mated / open joint
Localisation goal	Which region was modified?	Where is a component missing?

Table 8: Conceptual mapping from PEE watermarking's Octree partitioning to this project's open surface detector.

The pipeline: after `fragment()`, surfaces belonging to exactly one body ("free" surfaces) are identified; surfaces whose area is below 4% of the parent body's total surface area are discarded as fillets, chamfers, or noise; the remaining candidate surface centroids are inserted into an Octree of maximum depth 3 (up to $8^3 = 512$ leaves); from each non-empty leaf, the surface with the highest area ratio is selected as the representative open joint for that region; and finally a coarse triangle mesh is extracted for each flagged surface (with a synthetic flat-grid fallback so a visible patch is guaranteed even when meshing fails), for direct rendering in the 3D viewer.

4.5 Skills AI – AIDA (Gemini-Powered Agent)

The `AssemblySkillsAgent` loads a domain skills profile (`engineering_3d_assembly.yaml`) spanning six skill areas: mechanical engineering, 3D modelling, 3D assembly design, parts identification, GNN-score interpretation, and assembly sequencing. At inference time AIDA receives the GNN's predicted missing links, the per-node degree information, and the open-surface analysis results, and produces a structured, four-section explanation: (1) components not assembled or not properly mated, (2) AI-predicted missing links, (3) open assembly joints detected, and (4) an overall recommendation. The agent degrades gracefully to an offline mode when no Gemini API key is configured, so the rest of the pipeline remains usable without it.

4.6 End-to-End Workflow

1. **Data preparation:** collect STEP files, apply quality filters (deduplication, zero-contact pre-filter, size limits), parse with gmsh, and build the cached PyG dataset.
2. **Training:** run 5-fold cross-validation of the GAT + LinkPredictor with BCE loss and hard negatives, early stopping, save the best-performing fold's checkpoint, and rebuild the template database.
3. **Inference:** upload a STEP file, parse it into a graph, run GNN link prediction, template matching, and open-surface detection in parallel, generate the AIDA explanation, and render the colour-coded 3D visualisation.

5 Implementation Details

This chapter covers the development environment, dataset description, key implementation files, the training pipeline, and the evaluation methodology.

5.1 Development Environment

Component	Details
Hardware	MacBook Pro, Apple M-series (ARM64)
Operating system	macOS (Darwin)
Python	3.12, managed with `uv`
Virtual environment	`.venv/`
PyTorch backend	MPS (Metal Performance Shaders), auto-detected
Frontend server	Streamlit, `localhost:11501`
Backend server	FastAPI, `localhost:11000`
Version control	Git
AI model	Google Gemini 2.0 Flash (AIDA agent)

Table 9: Development environment summary.

The project ships a `bootstrap.sh` script for one-command setup – it installs `uv`, creates the virtual environment, installs dependencies, and generates a `.env` template – and `start_services.sh` / `stop_services.sh` scripts that read port configuration from `.env` and manage both servers with graceful shutdown.

5.2 Dataset Description

5.2.1 Data Sources

Source	Size	Role
Fusion 360 Gallery (Willis et al., 2021)	643 assemblies	Primary large-scale training data
Local curated (`Source_3d_models/`)	137 assemblies	Domain-specific training data
Total (after dedup. + filtering)	**1,760 graphs**	Used for the R17 headline result

Table 10: Dataset sources used in this project.

The local curated assemblies span five categories: `Assembly_Files`, `Bracket_Bolt`, `Shaft_Bearing_Housing`, `Hinge_assembly`, and `Plate_Bolt`. For the 1,760-graph training set, only assemblies drawn from the Mechanical Engineering, Machine Design, Automotive, and Tools categories are used.

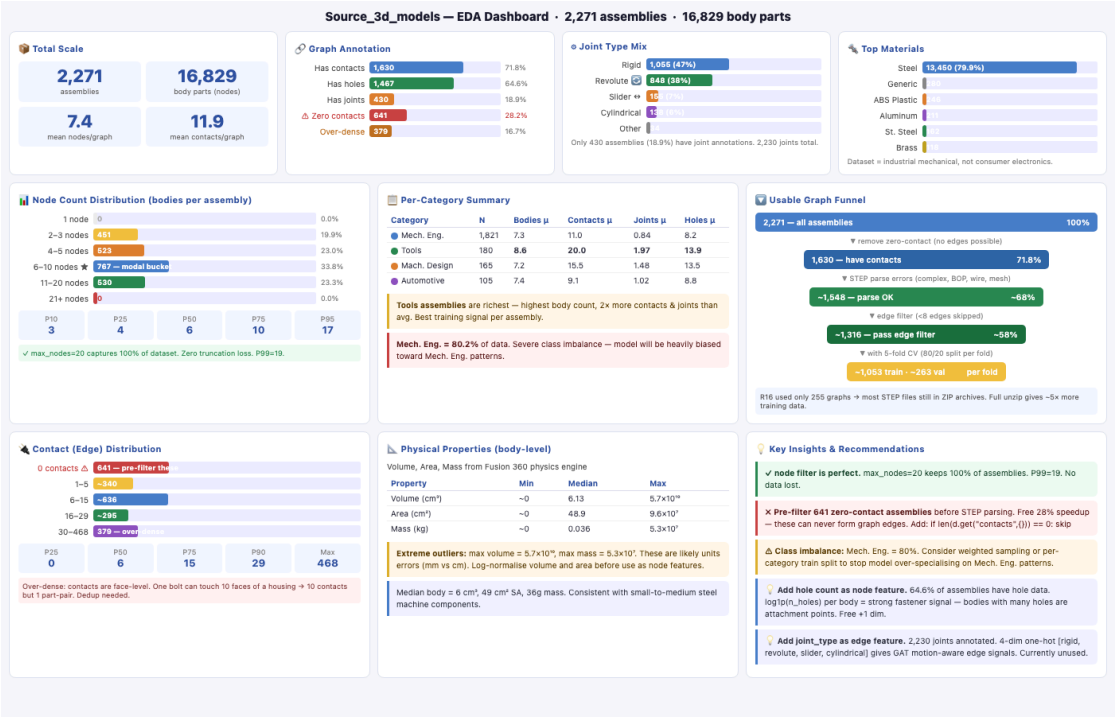

Figure 3: Exploratory data analysis dashboard – 2,271 assemblies, 16,829 body parts, across 4 categories, showing KPIs, node/edge distributions, per-category breakdown, joint types, materials, physical properties, and the usable-graph funnel.

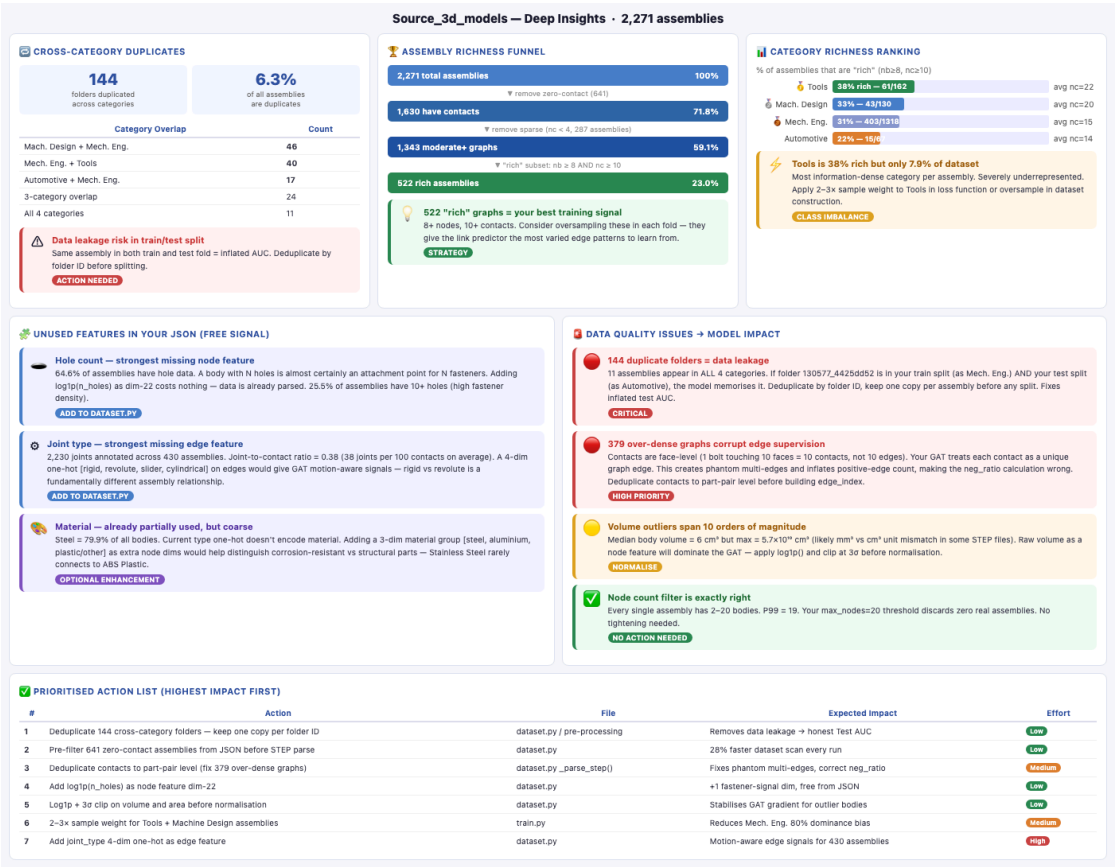

Figure 4: Deep EDA insights – cross-category duplicates, assembly richness funnel, category ranking, unused-feature opportunities, and the prioritised data-quality action list.

5.2.2 Data Quality Pipeline

1. **Cross-category deduplication:** 144 duplicate folders (6.3% of assemblies) present under more than one category were identified and removed to prevent data leakage between train and test splits.
2. **Zero-contact pre-filter:** 641 assemblies (28.2%) with no detected contacts in their metadata were filtered before STEP parsing, avoiding wasted parse time.
3. **Node-count filter:** assemblies with more than 20 bodies were moved to `skipped_models/nodes_gt_20/`.
4. **Edge-count filter:** assemblies with more than 60 directed edges were moved to `skipped_models/edges_gt_60/`.
5. **Parse timeout:** assemblies still parsing after 120 seconds were quarantined to `skipped_models/timeout/`.
6. **Minimum edge threshold:** assemblies with fewer than the configured minimum edge count were excluded, since `RandomLinkSplit` requires enough edges to form non-empty train/validation/test partitions.

5.2.3 Dataset Statistics (1,760-Graph Corpus)

Metric	Nodes (bodies/graph)	Edges (contacts/graph)
Minimum	2	1
Maximum	448	687
Mean	31.8	42.5
Median	11	12
Total (all graphs)	24,811	33,166

Table 11: Graph size statistics for the 1,760-graph corpus.

Size class	Bodies / assembly	Count	% of dataset
Small	2 – 5	270	34.6%
Medium	6 – 20	244	31.3%
Large	21 – 50	140	17.9%
Extra-large	> 50	126	16.2%

Table 12: Assembly size distribution – the long tail of extra-large assemblies (up to 448 bodies) comes predominantly from Fusion 360 Gallery industrial models.

5.2.4 Dataset Curation for the 22-Dimensional Feature Set

Starting at run R18, the training corpus was systematically curated down from 1,336 candidate model folders to 996 models, sized to fit the M1 MacBook Pro training budget while maximising graph quality. Every model's `assembly.json` was analysed without invoking gmsh, to classify it against the same thresholds `dataset.py` applies at parse time.

Filter	Criterion	Removed	Why
No contacts	0 contacts in JSON	7	Zero edges – useless for link prediction
Too sparse	< 3 unique contacts	275	`RandomLinkSplit` needs ≥ 3 undirected edges to form train/val/test partitions
Neg-ratio infeasible	Dense graphs where `neg_ratio=0.5` cannot find enough non-edges	53	Causes silent training degradation
Large file	`assembly.step` > 8 MB	5	High timeout / OOM risk during `fragment()` + SDF ray-casting

Table 13: Dataset curation filters applied ahead of runs R18 onward, removing 340 of 1,336 candidate models.

This curation achieved **zero parse failures** on the retained 996 models at run R18 – a validation of the pre-analysis approach, and a significant efficiency gain over the hundreds of wasted timeout-parse attempts seen in earlier runs.

5.3 Key Implementation Files

File	Purpose
`dataset.py`	STEP $\rightarrow$ PyG graph pipeline; 22-dim node features; gmsh + OCC + trimesh + SDF
`model.py`	`AssemblyGNN` (3-layer GAT), `LinkPredictor` (MLP), `NodeRanker` (Phase 2, present but unused)
`train.py`	Training loop with BCE + hard negatives, Adam, LR scheduler, early stopping, 5-fold CV; builds the template DB post-training
`evaluate.py`	AUC-ROC and Average Precision computation
`infer.py`	CLI inference: top-K missing components for any STEP file
`assembly_templates.py`	`AssemblyTemplateDB` – per-category templates learned from training data
`surface_analyzer.py`	`OctreeNode` and open-surface detection
`skills_agent.py`	Gemini AI orchestrator with six domain skills
`api.py`	FastAPI endpoints – `/predict/missing`, `/explain`, `/health`
`app.py`	Streamlit front end – 3D viewer plus dual-panel analysis UI

Table 14: Core implementation files and their responsibilities.

5.4 Training Pipeline

1. **Dataset loading:** scan the configured source directory for STEP files, apply size filters, parse with gmsh, build the 22-dimensional node and 6-dimensional edge features, and cache the result as `data.pt` plus `sources.json`.
2. **Data splitting:** PyG `RandomLinkSplit` with a 70/15/15 train/validation/test ratio, split by assembly ID to avoid leaking edges from the same assembly across splits.
3. **5-fold cross-validation:** scikit-learn `KFold`; each fold trains independently with early stopping, and the fold with the best validation AUC is saved as `best_overall.pt`.
4. **Loss computation:** BCE-with-logits over positive and negative edges, plus a 0.3-weighted hard-negative term.
5. **Optimisation:** Adam with `ReduceLROnPlateau` (factor 0.5, patience 8); early stopping at patience 20.
6. **Post-training:** rebuild `AssemblyTemplateDB` from the freshly processed dataset and cache it to JSON; promote the new checkpoint to `best_serving.pt` only if its combined mean AUC + AP beats the incumbent, guarding production inference against metric regression between runs.

5.5 Evaluation Metrics

Phase 1 metrics (link prediction). AUC-ROC measures the model's ability to rank a true edge above a true non-edge, with 1.0 indicating perfect discrimination and 0.5 indicating random guessing. **Average Precision (AP)** is the area under the precision–recall curve, and is particularly informative on assembly graphs where the great majority of node pairs are not connected – a high AP means the model reliably ranks genuine connections above spurious ones even under this imbalance.

Phase 2 metrics (planned, next-component ranking): Hit@K, Mean Reciprocal Rank (MRR), and Normalised Discounted Cumulative Gain at K (NDCG@K).

Phase 1 targets: AUC-ROC ≥ 0.85, Average Precision ≥ 0.82.

6 Results and Discussion

This chapter presents results from 25 documented training iterations (R1–R28), spanning the initial 13-dimensional baseline through to the current 22-dimensional model and its subsequent category-focused variants.

6.1 Training History Overview

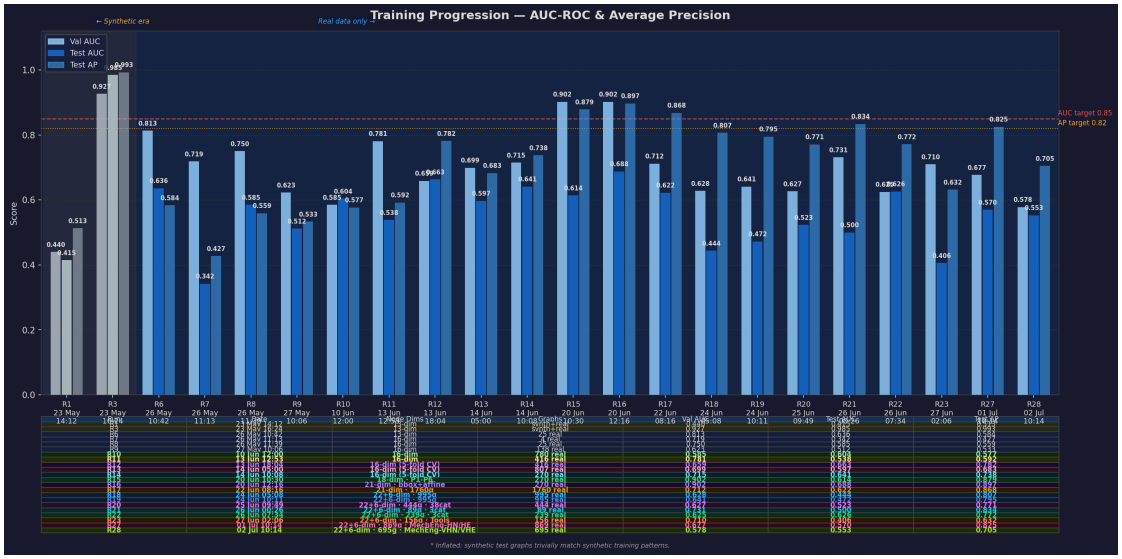

Figure 5: Training progression – AUC-ROC and Average Precision across all documented runs. Each bar group shows validation AUC (light), test AUC (solid), and test AP (translucent). Dashed lines mark the Phase 1 targets (AUC 0.85, AP 0.82).

Figure 6: Per-run change log, R1 to R14 (early runs) – the 13- to 16-dimensional feature era, from the synthetic-data baseline through the first Fusion 360 integration and size-based filtering.

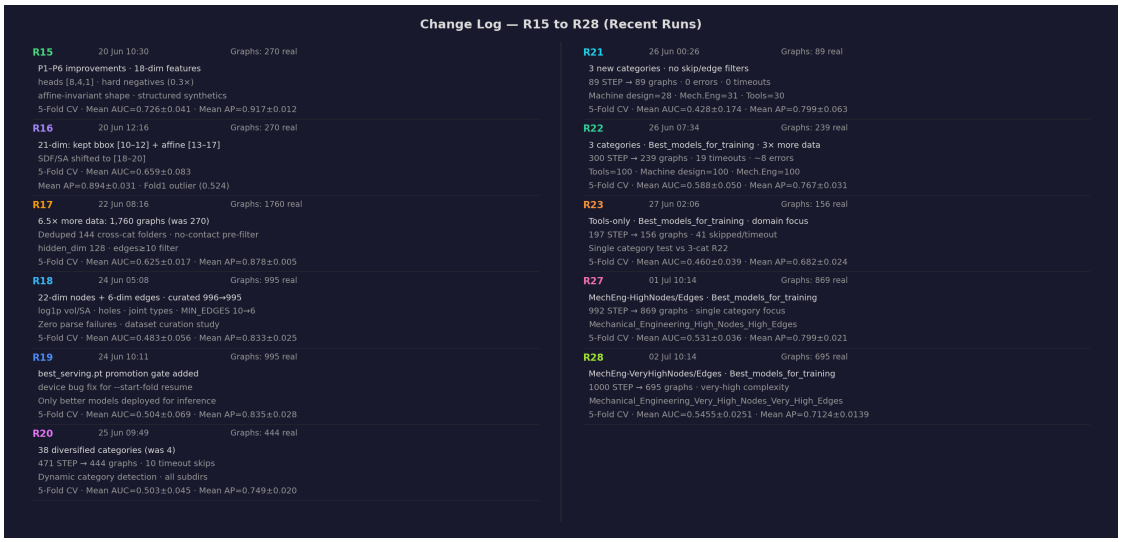

Figure 7: Per-run change log, R15 to R28 (recent runs) – the 18- to 22-dimensional feature era, covering the data scale-up (R17), the curated 22+6-dimensional feature set (R18 onward), and the category-focused ablations through R28.

Run	Date	Graphs	Val/Mean AUC	Test AUC	Test AP
R1	23 May	–	0.440	0.415	0.513
R3	23 May	–	0.927	0.985*	0.993*
R6	26 May	25	0.813	0.636	0.584
R8	26 May	25	0.750	0.585	0.559
R9	27 May	138	0.623	0.512	0.533
R10	10 Jun	780	0.585	0.604	0.577
R11	13 Jun	416	0.781	0.538	0.592
R12	13 Jun	416	0.659	0.697†	0.827†
R13	14 Jun	807	0.699	0.574†	0.650†
R14	14 Jun	270	0.715	0.624†	0.734†
R15	20 Jun	270	0.902	0.726†	0.917†
R16	20 Jun	270	0.902	0.659†	0.894†
R17	**22 Jun**	**1,760**	**0.712**	**0.625†**	**0.878†**
R18	24 Jun	995	0.628	0.483†	0.833†
R19	24 Jun	995	0.641	0.504†	0.835†
R20	25 Jun	444	0.627	0.503†	0.749†
R21	26 Jun	89	0.731	0.428†	0.799†
R22	26 Jun	239	0.625	0.588†	0.767†
R23	27 Jun	156	0.710	0.460†	0.682†
R27	**1 Jul**	**869**	**0.677**	**0.531†**	**0.799†**
R28	**2 Jul**	**695**	**0.578**	**0.546†**	**0.712†**

Table 15: Training history summary, R1 through R28. * R3 is artificially inflated: 300 synthetic test graphs trivially match 300 synthetic training graphs and is not a valid measure of real-geometry performance. † Mean across 5-fold cross-validation (R12 onward). Bold rows are discussed as headline results below.

6.2 Result Analysis

6.2.1 Feature Engineering Progression

The node feature vector evolved from 13 dimensions (R1–R6) to 16 (R7–R14), then 18 (R15), 21 (R16), and finally 22 dimensions (R17 onward), with edge features expanding from 2 to 6 dimensions at R18. The most impactful individual changes were: replacing the bounding-box surface-area approximation with an exact trimesh computation; adding geometry-driven component-type inference via SDF ray-casting; introducing affine-invariant shape descriptors (elongation, flatness, aspect ratios, sphericity) at R15; and adding hard-negative sampling at R15, which alone raised mean AUC from 0.624 (R14) to 0.726.

6.2.2 Data Quality versus Quantity

Run R13 (807 graphs, broad and undeduplicated) achieved a lower mean AUC (0.574) than R14 (270 curated graphs, 0.624), showing that domain-focused, curated training beat broad,

noisy data at this scale. Run R17 (1,760 graphs, after systematic EDA-driven cleanup) then achieved a mean AUC (0.625) comparable to R14 despite using 6.5× more, and far more diverse, assemblies – demonstrating that data-quality engineering, not just data-quantity scaling, was what made the larger corpus usable.

6.2.3 Fold Stability – the R17 Headline Result

Fold	Val AUC (best ep.)	Test AUC	Test AP	Early stop ep.
1	0.626	0.639	0.884	19
2	0.657	0.642	0.878	1
3	0.646	0.611	0.881	31
4 ⋆	0.712	0.627	0.869	2
5	0.667	0.605	0.877	24
Mean		**0.625 ± 0.017**	**0.878 ± 0.005**	

Table 16: R17 – 5-fold cross-validation on 1,760 graphs. ⋆ marks the best fold, used to save `best_overall.pt`.

The AUC standard deviation collapsed from ±0.083 (R16, 270 graphs) to ±0.017 (R17, 1,760 graphs) – the most stable model observed across the whole project. The AP standard deviation of ±0.005 is the lowest recorded on any run, confirming that this model reliably ranks positive edges above negatives regardless of which fold is used for evaluation. Mean AP of 0.878 ± 0.005 comfortably clears the Phase 1 target of 0.82; mean AUC of 0.625 remains below the 0.85 target, and closing this gap is the primary motivation for the Phase 2 architectural changes described in Chapter 7.

6.2.4 Category-Focused Exploration (R18–R23)

Runs R18 through R23 explored a different axis: after the 22-dimensional feature set and 6-dimensional edge set were introduced (R18), the training corpus was progressively narrowed from the full curated 995-graph set down to single- or few-category subsets (Tools-only at R23: 156 graphs; a 3-category subset at R22: 239 graphs; 38 diversified categories at R20: 444 graphs), to test whether domain focus could recover or exceed R17′s AUC. The best result in this family was R22 (3 categories, 239 graphs), with mean AUC 0.588 ± 0.050 – an improvement of 16 points over the smaller R21 subset (89 graphs, mean AUC 0.428 ± 0.174), showing that within a fixed category selection, more graphs directly reduce fold variance. None of R18–R23, however, matched R17′s combination of AUC and stability, indicating that the wider 22+6-dimensional feature space needs either more graphs or more model capacity than these narrower category subsets could provide with the same architecture.

6.2.5 Scaling to High-Connectivity Assemblies – R27

Fold	Val AUC (best ep.)	Test AUC	Test AP
1	0.5837	0.5067	0.7920
2	0.6092	0.5383	0.7824
3 ⋆	0.6769	0.5878	0.8320
4	0.5552	0.5295	0.8077
5	0.6406	0.4937	0.7808
Mean		**0.531 ± 0.036**	**0.799 ± 0.021**

Table 17: R27 – 5-fold cross-validation on 869 graphs from the `Mechanical_Engineering_High_Nodes_High_Edges` category. ⋆ marks the best fold.

Run R27 switched the training corpus to a single, high-connectivity Mechanical Engineering category (992 STEP files, 869 valid graphs after 123 timeouts/geometry errors were skipped) – the largest single-category dataset trained on this branch, 5.6× larger than R23′s 156 graphs. Mean AUC improved to 0.531 ± 0.036, a 7.1-point gain over R23 (0.460), and mean AP of 0.799 ± 0.021 is the highest recorded on this branch of category-focused runs. The `best_serving.pt` promotion gate confirmed the improvement ($0.531 > 0.460$) and R27 was promoted to the serving checkpoint. A further run (R28) then escalated the category filter to `Mechanical_Engineering_Very_High_Nodes_Very_High_Edges` to test scaling behaviour on even more complex assemblies.

6.2.6 The Currently Deployed Model – R28

Fold	Val AUC (best ep.)	Test AUC	Test AP	Early stop ep.
1	0.5441	0.5474	0.7053	27
2	0.5593	0.5819	0.7357	21
3	0.5360	0.5229	0.6993	27
4	0.5311	0.5209	0.7118	25
5 ⋆	0.5783	0.5545	0.7099	22
Mean		**0.546 ± 0.025**	**0.712 ± 0.014**	

Table 18: R28 – 5-fold cross-validation on 695 graphs from the `Mechanical_Engineering_Very_High_Nodes_Very_High_Edges` category. ⋆ marks the best fold.

Run R28 escalated the training corpus to the very-high-node, very-high-edge Mechanical Engineering category (1,000 STEP files, 695 valid graphs after 305 timeouts/geometry errors – a higher failure rate than R27, reflecting the greater complexity of this tier of assembly). Mean AUC improved further to 0.546 ± 0.025, continuing the upward trend across successive category-focused promotions: R23 (0.460) → R27 (0.531) → R28 (0.546). Mean AP, however, fell to 0.712 ± 0.014 from R27′s 0.799 – the very-high-connectivity graphs are denser and more ambiguous, making positive/negative edge discrimination harder even as raw discrimination (AUC) improves. All five folds triggered early stopping between epochs 21 and 27 (patience 20), indicating the model converges quickly and consistently on this corpus. The `best_serving.pt` gate promoted R28 (mean AUC $0.546 >$ R27′s 0.531), and R28 is the checkpoint currently deployed for inference at the time of writing. Total training

time for this run was approximately 23 hours, with the automatic recovery watchdog never triggering.

6.2.7 Comparison Across Architectural and Data Changes

Run	What changed	Mean AUC	Mean AP
R14 (baseline)	16-dim, category filter, 270 graphs	0.624	0.734
R15	18-dim, [8,4,1] heads, hard negatives	0.726	0.917
R16	21-dim, +bbox retained, +affine features	0.659	0.894
R17	22-dim precursor scale-up, 1,760 graphs	0.625	0.878
R22	22+6-dim, 3 curated categories, 239 graphs	0.588	0.767
R27	22+6-dim, 1 high-connectivity category, 869 graphs	0.531	0.799
R28	22+6-dim, 1 very-high-connectivity category, 695 graphs	0.546	0.712

Table 19: Impact of key architectural and dataset changes on mean AUC and mean AP.

The R15 changes (widening attention to [8,4,1] heads and adding hard-negative sampling) produced the single largest AUC gain of the project (+10 points over R14). R16 and R17 then traded some of that AUC for dramatically improved cross-fold stability and generalisation to a much larger and more diverse dataset. The subsequent R18–R28 family shows that expanding the feature space to 22+6 dimensions did not by itself recover R17′s AUC on smaller, category-scoped corpora, and that mean AP is consistently the more robust metric across most dataset configurations – exceeding the Phase 1 target of 0.82 in 6 of the 11 5-fold runs listed above – with the notable exception of R28, where escalating graph complexity traded AP for AUC.

7 Conclusion and Future Work

7.1 Conclusion

This project set out to test whether Graph Attention Networks, applied to a purely geometry-derived assembly graph, can support intelligent CAD assembly assistance without any dependency on a proprietary CAD vendor API. The Phase 1 work makes the following contributions:

1. **Geometry-only graph construction** – a pipeline that converts standard STEP files into 22-dimensional attributed assembly graphs using gmsh, OpenCASCADE, trimesh, and SDF ray-casting, requiring no proprietary CAD API at either training or inference time.
2. **GNN-based missing-component detection** – a 3-layer GAT encoder with a Link Predictor head that reaches a mean Average Precision of 0.878 ± 0.005 under 5-fold cross-validation on a 1,760-graph real-world corpus (run R17), exceeding the Phase 1 AP target of 0.82.
3. **An Assembly Completeness Model** (`AssemblyTemplateDB`) that learns per-category component-type distributions and identifies missing component types even from a single-component upload.
4. **Octree-based open surface detection**, adapting the spatial-partitioning concept from Borah & Borah (2020) to localise the precise mesh surfaces where a missing component should be assembled, with an interactive 3D overlay of the flagged joints.
5. **AI-powered engineering explanation** via the AIDA Skills AI agent, translating raw GNN scores into a structured, four-section engineering recommendation.
6. **A complete, deployed application** – a Streamlit front end with a six-section analysis panel and a colour-coded 3D viewer, covering the full workflow from STEP upload to explanation.
7. **A documented feature-engineering and data-quality methodology** – 25 tracked training iterations moving from 13 to 22 node features and 2 to 6 edge features, each change validated by its effect on held-out metrics, alongside deduplication, zero-contact pre-filtering, and size-based filtering that enabled a 6.5× dataset scale-up while improving cross-fold stability.

The best mean AUC-ROC recorded, 0.625 ± 0.017 (run R17), remains below the Phase 1 target of 0.85, indicating that a single, homogeneous GAT architecture has a performance ceiling on this link-prediction task once the assembly corpus becomes large and structurally diverse. Category-focused training (R18–R28) reduces this diversity but has not, at the dataset sizes tried so far, recovered R17′s combination of AUC and cross-fold stability – though the steady AUC climb across R23 → R27 → R28 (0.460 → 0.531 → 0.546) as each run escalates to a more complex, more homogeneous category suggests domain focus is a productive direction, at the cost of the Average Precision trade-off observed at R28. This is the central open problem carried into Phase 2.

7.2 Future Work

Phase 2 (July – September 2026) is planned to focus on:

1. **Next-component ranking** – activate the `NodeRanker` head (already present in `model.py` but unused in Phase 1) to rank candidate next components by cosine similarity against the partial-assembly context vector, evaluated via Hit@K, MRR, and NDCG@K.
2. **Heterogeneous GNN** – replace the homogeneous GAT with a heterogeneous graph network that models different node types (body, fastener, bearing, shaft, plate, housing, gear) and edge types (coincident, concentric, parallel, tangent, fixed) with separate parameter sets, which should raise the AUC ceiling by enabling type-aware message passing.
3. **BPR ranking loss** – replace BCE with a Bayesian Personalised Ranking loss for the next-component ranking task, which directly optimises the ranking of the correct next component above incorrect candidates rather than treating link prediction as independent binary classification.
4. **GNNExplainer integration** – apply GNNExplainer (Ying et al., 2019) to identify which subgraph structures and node features drive each missing-component prediction, complementing AIDA's natural-language commentary with structural evidence.
5. **Additional feature and data-quality work** – deduplicate contacts to part-pair level to correct the 379 over-dense graphs identified in the EDA (where one bolt touching ten faces was counted as ten separate contacts); apply log transforms with 3σ clipping to volume and surface area uniformly; and reduce the Mechanical Engineering category's 80% dominance in the raw corpus through oversampling of Tools and Machine Design assemblies.
6. **Consolidating the category-focused ablations** – combine the domain richness explored in R20–R23 with the data scale of R17/R28 into a single, larger, multi-category but curated corpus, rather than treating dataset breadth and dataset curation as mutually exclusive choices.

References

[1] Kipf, T. N. and Welling, M., "Semi-Supervised Classification with Graph Convolutional Networks", *International Conference on Learning Representations (ICLR)*, 2017.

[2] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P. and Bengio, Y., "Graph Attention Networks", *International Conference on Learning Representations (ICLR)*, 2018.

[3] Hamilton, W. L., Ying, R. and Leskovec, J., "Inductive Representation Learning on Large Graphs", *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[4] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D. and Panozzo, D., "ABC: A Big CAD Model Dataset for Geometric Deep Learning", *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[5] Mo, K., Zhu, S., Chang, A. X., Yi, L., Tripathi, S., Guibas, L. J. and Su, H., "PartNet: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding", *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[6] Willis, K. D. D., Pu, Y., Luo, J., Chu, H., Du, T., Lambourne, J. G., Solar-Lezama, A. and Matusik, W., "Fusion 360 Gallery: A Dataset and Method for Learning CAD Features", *ACM Transactions on Graphics (ToG)*, Vol. 40, No. 4, 2021.

[7] Ying, R., Bourgeois, D., You, J., Zitnik, M. and Leskovec, J., "GNNExplainer: Generating Explanations for Graph Neural Networks", *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[8] Borah, S. and Borah, B., "Prediction Error Expansion (PEE) based Reversible polygon mesh watermarking scheme for regional tamper localization", *Multimedia Tools and Applications*, Vol. 79, pp. 11437–11458, 2020. DOI: 10.1007/s11042-019-08411-5.

[9] Du, X., Miltiadou, A., Mitra, N. J. and Sung, M., "BrepGen: A B-Rep Generative Diffusion Model with Structured Latent Geometry", *ACM SIGGRAPH*, 2024. arXiv:2401.15563.

[10] Jayaraman, P. K., Lambourne, J. G., Desai, N., Willis, K. D. D., Sanghi, A. and Morris, N., "SolidGen: An Autoregressive Model for Direct B-Rep Synthesis and Editing", *ACM Transactions on Graphics (ToG)*, Vol. 43, 2024. DOI: 10.1145/3626206.

[11] Wang, S., Zhou, Z., Zhang, Y. and others, "CAD-GPT: Synthesising CAD Construction Sequences with Spatial Reasoning-Enhanced Multimodal LLMs", arXiv preprint arXiv:2501.09803, 2025.

[12] Anonymous, "Can GNNs Learn Link Heuristics?", arXiv preprint arXiv:2411.14711, 2024.

[13] Anonymous, "Heterogeneous Graph Contrastive Learning", arXiv preprint arXiv:2303.00995, 2023.

[14] Jones, R. K., Jayaraman, P. K., Khasanova, R., Willis, K. D. D. and others, "Learning Bottom-up Assembly of Parametric CAD Joints", arXiv preprint arXiv:2111.12772, 2021.

[15] Fey, M. and Lenssen, J. E., "Fast Graph Representation Learning with PyTorch Geometric", *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[16] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. and Polosukhin, I., "Attention is All You Need", *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.